

GOLDEN HOUSE

Solution M-Commerce de Mobilier & Décoration d'Intérieur

RAPPORT D'ARCHITECTURE MICROSERVICES

Découpage des Composants, Matrice des Communications & Modèles de Données

Équipe Projet :

Justin CHAPON
Nabil BENNACER
Tom CANTILLON
Salma EL ANNABY
Manuela ZAPATA QUIRÓS

Cadre Pédagogique :

Module : **IWA 2** (Web Avancé)
Enseignant : Christophe NAUROY
Polytech Montpellier – IG5 / DaMS
Semestre 9 – Année 2026-2027

Objet du livrable : Spécification formelle de l'architecture distribuée Spring Boot pour Golden House. Ce document synthétise le découpage en microservices, justifie la nature synchrone ou asynchrone de chaque flux, intègre formellement la composante de **Facturation**, et documente pour chaque service ses points d'entrée (APIs REST) et son schéma de données PostgreSQL.

Septembre 2026

Table des matières

1	Découpage de l'Architecture Microservices	1
1.1	Contexte et Approche d'Urbanisation	1
1.2	Intégration du Service de Facturation (Billing Service)	1
1.3	Cartographie Globale des Microservices et Flux	2
2	Matrice des Communications : Synchrone vs Asynchrone	3
2.1	Critère d'Urbanisation et Règle de Décision	3
2.2	Matrice Exhaustive des Flux Inter-Services	3
3	Points d'Entrée et Modèles de Données par Microservice	5
3.1	User Service	5
3.2	Catalog Service	5
3.3	Cart Service	6
3.4	Order Service	6
3.5	Payment Service	7
3.6	Billing Service (Facturation)	7
3.7	Notification Service	8
3.8	Review Service	8
3.9	Supplier Service	9
3.10	Analytics & IA Batches (Spark)	9
4	Synthèse et Alignement avec le Projet	11

1. Découpage de l'Architecture Microservices

1.1. Contexte et Approche d'Urbanisation

La solution m-commerce **Golden House** assure la vente directe d'articles de mobilier et de décoration (canapés, lits, rideaux, coussins). Afin d'offrir une expérience utilisateur fluide sur mobile, une gestion réactive pour les commerçants et une haute tolérance aux pannes, le backend adopte une **architecture orientée microservices** articulée autour du framework **Spring Boot** et d'une base relationnelle **PostgreSQL**.

L'architecture globale se structure en 5 couches fonctionnelles :

1. Interfaces Utilisateurs (Frontends) :

- **App Mobile Acheteur (React Native)** : Navigation, panier, paiement sécurisé, suivi de commande et avis.
- **Backoffice Web Commerçant (React)** : Dashboard KPI, gestion du stock, commandes fournisseurs et modération.
- **Chatbot IA** : Assistant conversationnel client s'appuyant sur des agents et le protocole standardisé MCP.

2. Passerelles d'Accès (Gateways) :

- **API Gateway (Spring Cloud Gateway)** : Point d'entrée unique, routage dynamique et validation des jetons JWT.
- **MCP Gateway (Model Context Protocol)** : Passerelle dédiée aux agents IA pour requêter le catalogue et les métriques.

3. Microservices Métier (Spring Boot) : Neuf services autonomes et hautement cohésifs :

- **User Service** : Comptes, profils, adresses et authentification JWT.
- **Catalog Service** : Produits, filtres de recherche, stocks et seuils d'alerte.
- **Cart Service** : Panier temporaire d'achat, calculs de totaux et codes de promotion.
- **Order Service** : Enregistrement, cycle de vie et suivi des commandes.
- **Payment Service** : Passerelle Stripe (PaymentIntent, sessions et webhooks).
- **Billing Service (Facturation)** : Pièces comptables légales immuables et factures PDF.
- **Notification Service** : Alertes stock bas et notifications d'état par push/email.
- **Review Service** : Notes et avis clients, réponses du commerçant.
- **Supplier Service** : Fournisseurs, commandes d'achat et réceptions de stock.

4. Traitements Batch Analytiques & IA (Spark) :

- **KPI Batch (Spark Java)** : Calcul différé des métriques (CA global/produit, produit star, rupture).
- **Recommendation Batch (Spark MLlib – ALS)** : Filtrage collaboratif calculant les similarités produits.

5. Persistance & Services Tiers :

- **PostgreSQL** : Pivot relationnel assurant l'intégrité transactionnelle ACID des ventes.
- **Stripe API** : Passerelle bancaire externe pour l'exécution conforme et sécurisée des paiements.

1.2. Intégration du Service de Facturation (Billing Service)

L'analyse des dépendances du système a permis d'isoler la **Facturation** au sein d'un microservice dédié (*Billing Service*) :

- **Exigence légale & fiscale** : Une facture est une pièce comptable immuable, possédant une numérotation séquentielle stricte et ventilant les taux de TVA. Elle ne doit pas être altérée si la commande évolue ultérieurement.
 - **Isolation des traitements lourds** : La compilation des fichiers PDF et leur archivage dans un stockage persistant constituent des opérations lourdes qui ne doivent jamais dégrader le temps de réponse du checkout.
 - **Découplage asynchrone** : Le *Billing Service* écoute l'événement de validation bancaire, génère la pièce comptable en tâche de fond et déclenche son expédition par le *Notification Service*.
-

1.3. Cartographie Globale des Microservices et Flux

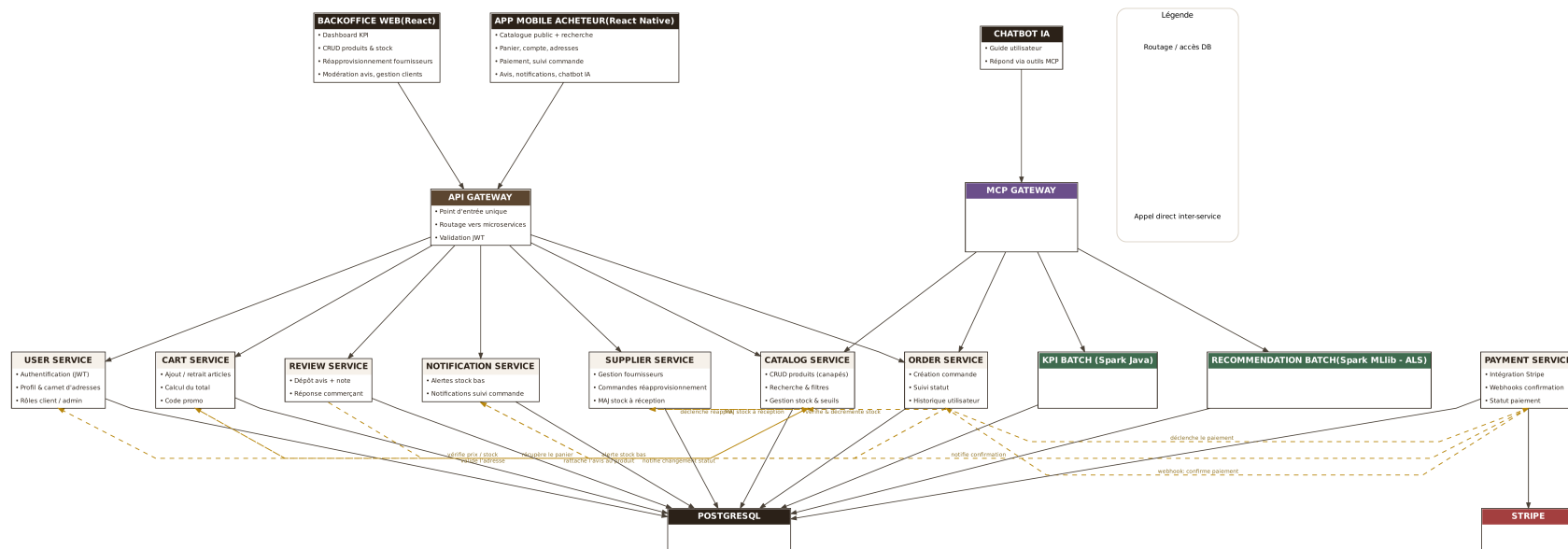


FIGURE 1 – Architecture globale des microservices de Golden House (Flux d'accès, passerelles et dépendances inter-services)

Légende des communications :

- **Trait plein noir** : Routage des requêtes HTTP externes par l'API Gateway ou requêtes d'outils par le serveur MCP Gateway.
- **Trait pointillé or/marron** : Appels directs inter-services (dépendances métier détaillées à la section 1.2).
- **Accès direct DB (flèches vers PostgreSQL)** : Persistance relationnelle propre à chaque domaine métier.

2. Matrice des Communications : Synchrone vs Asynchrone

2.1. Critère d'Urbanisation et Règle de Décision

Pour chaque interaction inter-services, le choix du mode de transport repose sur une règle claire :

Règle d'or d'urbanisation : Synchrone vs Asynchrone

- **Mode Synchrone (HTTP / REST)** : L'appelant a besoin de la réponse **immédiatement** pour continuer son exécution, prendre une décision bloquante ou répondre sans délai à l'utilisateur. Toute indisponibilité du service appelé empêche l'opération de se conclure.
- **Mode Asynchrone (Events / Message Broker / Webhooks)** : L'appelant peut libérer l'utilisateur **sans attendre** l'achèvement de l'action. La propagation de l'effet en tâche de fond (quelques centaines de millisecondes ou secondes) préserve la cohérence métier à terme (*eventual consistency*) et isole les pannes.

2.2. Matrice Exhaustive des Flux Inter-Services

Flux (Émetteur → Cible)	Nature de l'échange	Mode	Justification Technique
Cart → Catalog	Vérification prix et stock	Synchrone	L'acheteur clique sur « Ajouter au panier ». Le Cart Service doit savoir instantanément si le produit existe et si le stock est suffisant.
Order → Cart	Récupération du panier	Synchrone	Au checkout, la commande a impérativement besoin du contenu exact du panier pour créer l'ordre d'achat.
Order → Catalog	Décrémentation du stock	Synchrone	Prévention absolue de la survente (<i>overselling</i>) en cas d'achats simultanés sur la dernière pièce d'un meuble.
Order → User	Validation de l'adresse	Synchrone	Contrôle de sécurité immédiat : s'assurer que l'adresse de livraison appartient bien à l'utilisateur connecté.
Order → Payment	Initialisation session paiement	Synchrone	Order a besoin de l'identifiant de session Stripe (ou PaymentIntent) en retour immédiat pour rediriger le client.
Stripe → Payment	Webhook confirmation	Asynchrone	Événement externe envoyé par Stripe après validation de la carte bancaire. Personne n'attend en direct sur la ligne.
Payment → Order	Confirmation paiement	Asynchrone	Mise à jour résiliente du statut de la commande (PAYEE). Si Order est en maintenance, l'événement est rejoué sans perte.
Payment → Billing	Déclenchement facturation	Asynchrone	Dès paiement validé, la création de la facture légale et du PDF est effectuée en tâche de fond.
Payment → Notification	Confirmation de commande	Asynchrone	L'envoi du mail de confirmation est non bloquant. Une défaillance de notification ne doit pas annuler la vente.

Flux (Émetteur → Cible)	Nature de l'échange	Mode	Justification Technique
Billing → Notification	Transmission facture	Asynchrone	Notification avec pièce jointe ou lien de téléchargement envoyée dès que le PDF est compilé.
Order → Notification	Changement de statut	Asynchrone	Les alertes de suivi (préparation, expédition) sont notifiées au client sans impacter les outils logistiques.
Catalog → Notification	Alerte stock bas	Asynchrone	Si le stock passe sous le seuil d'alerte, notification envoyée au commerçant sans ralentir les consultations web.
Catalog → Supplier	Déclenchement réappro	Asynchrone	Le réapprovisionnement fournisseur s'inscrit sur un délai de plusieurs jours : découplage complet par message.
Supplier → Catalog	MAJ stock à réception	Asynchrone	Lors de la réception physique des colis, le stock est réincrémenté en tâche de fond dans le catalogue.
Review → Catalog	Vérification produit	Synchrone	Vérifie que l'article existe avant d'accepter le commentaire, évitant tout avis orphelin.
Review → Catalog	Mise à jour note moyenne	Asynchrone	Propagation différée de la note moyenne recalculée en cache sur la fiche produit pour préserver la vitesse de lecture.
Batches → PostgreSQL	Calculs analytiques	Asynchrone	Jobs Spark exécutés sur ordonnanceur planifié (cron) indépendamment des requêtes web.

3. Points d'Entrée et Modèles de Données par Microservice

Cette section formalise pour chaque composant sa responsabilité, ses points d'entrée (endpoints d'API REST et événements) ainsi que les tables PostgreSQL associées.

3.1. User Service

Responsabilité : Gestion des comptes utilisateurs, émission et validation des jetons JWT, gestion des rôles (Client, Commerçant, Admin), devises (€/ \$) et carnet d'adresses.

User Service – Spécification Technique

Points d'Entrée (APIs REST) :

- POST /api/v1/auth/register : Inscription client (body : email, password, username, phone).
- POST /api/v1/auth/login : Authentification et émission du token JWT.
- GET /api/v1/users/me : Consultation du profil connecté.
- PUT /api/v1/users/me : Modification des informations personnelles et de la devise.
- GET /api/v1/users/me/addresses : Récupération du carnet d'adresses.
- POST /api/v1/users/me/addresses : Ajout d'une adresse de livraison ou facturation.
- DELETE /api/v1/users/me/addresses/{id} : Suppression d'une adresse.

Modèle de Données Associé :

- **Table users** : id (UUID, PK), email (VARCHAR, UNIQUE), password_hash (VARCHAR), username (VARCHAR, UNIQUE), first_name (VARCHAR), last_name (VARCHAR), phone (VARCHAR), role (ENUM : 'CLIENT', 'COMMERÇANT', 'ADMIN'), currency (ENUM : 'EUR', 'USD'), stripe_customer_id (VARCHAR, UNIQUE), is_active (BOOLEAN), created_at (TIMESTAMP).
- **Table addresses** : id (UUID, PK), user_id (UUID, FK → users), type (ENUM : 'LIVRAISON', 'FACTURATION', 'MIXTE'), street (VARCHAR), complement (VARCHAR), postal_code (VARCHAR), city (VARCHAR), country (VARCHAR), is_default (BOOLEAN).

3.2. Catalog Service

Responsabilité : Gestion du catalogue d'ameublement, moteur de recherche et filtres par critères techniques, gestion des stocks en temps réel et détection du seuil d'alerte.

Catalog Service – Spécification Technique

Points d'Entrée (APIs REST & Événements) :

- GET /api/v1/products : Recherche filtrée (catégorie, matière, couleur, prix min/max, stock).
- GET /api/v1/products/{id} : Fiche article détaillée avec photos et dimensions.
- POST /api/v1/products : Création d'un produit (réservé rôle ADMIN/COMMERCEANT).
- PUT /api/v1/products/{id} : Mise à jour des caractéristiques techniques et prix.
- PATCH /api/v1/products/{id}/stock : Ajustement ou décrémentation atomique du stock.
- Événement consommé : SupplyReceivedEvent (incrément de stock depuis Supplier Service).

Modèle de Données Associé :

- Table products : id (UUID, PK), nom (VARCHAR), description (TEXT), prix (NUMERIC(10,2)), categorie (ENUM : CANAPE_DROIT, CANAPE_ANGLE, CANAPE_CONVERTIBLE, FAUTEUIL_LOVESEAT, MODULABLE, VINTAGE), type_matiere (ENUM : CUIR, VELOURS, TISSU_BOUCLETTE, LIN, MICROFIBRE, BOIS_MASSIF), couleur (VARCHAR), largeur_cm, hauteur_cm, profondeur_cm (INT), stock (INT), seuil_alerte (INT), images (TEXT[]), fournisseur_id (UUID, FK → fournisseurs), est_actif (BOOLEAN).

3.3. Cart Service

Responsabilité : Gestion des paniers clients, calcul des montants cumulés, application de coupons promotionnels et maintien du panier actif avant passage de commande.

Cart Service – Spécification Technique

Points d'Entrée (APIs REST) :

- GET /api/v1/carts/me : Consultation du panier de l'utilisateur connecté.
- POST /api/v1/carts/me/items : Ajout d'une ligne d'article (product_id, quantite).
- PATCH /api/v1/carts/me/items/{itemId} : Modification de la quantité choisie.
- DELETE /api/v1/carts/me/items/{itemId} : Suppression d'un article du panier.
- POST /api/v1/carts/me/promo : Saisie d'un code de réduction promotionnel.
- DELETE /api/v1/carts/me : Réinitialisation du panier après passage de commande.

Modèle de Données Associé :

- Table carts : id (UUID, PK), user_id (UUID, UNIQUE, FK → users), code_promo (VARCHAR), updated_at (TIMESTAMP).
- Table cart_items : id (UUID, PK), cart_id (UUID, FK → carts), product_id (UUID, FK → products), quantite (INT, CHECK > 0), contrainte d'unicité (cart_id, product_id).

3.4. Order Service

Responsabilité : Création de la commande, figeage du prix des articles au moment de l'achat, transitions de statuts (en attente, payée, en préparation, expédiée, livrée) et historique client.

Order Service – Spécification Technique

Points d'Entrée (APIs REST & Événements) :

- POST /api/v1/orders : Création d'une commande à partir du panier courant.
- GET /api/v1/orders/{id} : Détails complets d'une commande spécifique.
- GET /api/v1/orders/my-orders : Liste de l'historique d'achats du client.
- PATCH /api/v1/orders/{id}/status : Mise à jour du statut logistique par le commerçant.
- Événement consommé : PaymentConfirmedEvent (transition de statut vers PAYEE).

Modèle de Données Associé :

- **Table orders** : id (UUID, PK), order_number (VARCHAR, UNIQUE), user_id (UUID, FK → users), statut (ENUM : 'EN_ATTENTE_PAIEMENT', 'PAYEE', 'EN_PREPARATION', 'EXPEDIEE', 'LIVREE', 'ANNULEE'), statut_paiement (ENUM : 'PENDING', 'SUCCEEDED', 'FAILED', 'REFUNDED'), shipping_address_id (UUID, FK → addresses), billing_address_id (UUID, FK → addresses), montant_total (NUMERIC(10,2)), devise (ENUM : 'EUR', 'USD'), code_promo_applique (VARCHAR), created_at (TIMESTAMP).
- **Table order_items** : id (UUID, PK), order_id (UUID, FK → orders), product_id (UUID, FK → products), nom_produit (VARCHAR), quantite (INT), prix_unitaire (NUMERIC(10,2)), total_ligne (NUMERIC(10,2)).

3.5. Payment Service

Responsabilité : Communication avec l'API Stripe, création des sessions sécurisées Checkout / PaymentIntent, vérification des signatures et gestion des webhooks de paiement.

Payment Service – Spécification Technique

Points d'Entrée (APIs REST & Webhooks) :

- POST /api/v1/payments/create-session : Initialisation d'une session de paiement Stripe.
- POST /api/v1/payments/webhook : Réception des événements Stripe (payment_intent.succeeded).
- GET /api/v1/payments/status/{orderId} : Consultation du statut de la transaction.

Modèle de Données Associé :

- Champs Stripe rattachés à la commande : stripe_payment_intent_id (VARCHAR, UNIQUE), stripe_session_id (VARCHAR), statut_paiement (ENUM payment_status_enum).

3.6. Billing Service (Facturation)

Responsabilité : Émission des factures légales immuables, calcul détaillé des taxes (TVA), compilation déportée des documents PDF et mise à disposition pour téléchargement.

Billing Service – Spécification Technique**Points d'Entrée (APIs REST & Événements) :**

- GET /api/v1/invoices/order/{orderId} : Téléchargement du fichier PDF de la facture.
- GET /api/v1/invoices/my-invoices : Historique des factures de l'acheteur connecté.
- **Événement consommé** : OrderPaidEvent émis de façon asynchrone par Payment Service.

Modèle de Données Associé :

- **Table invoices** : id (UUID, PK), invoice_number (VARCHAR, UNIQUE, ex : INV-2026-00451), order_id (UUID, UNIQUE, FK → orders), user_id (UUID, FK → users), montant_ht (NUMERIC(10,2)), taux_tva (NUMERIC(4,2)), montant_tva (NUMERIC(10,2)), montant_ttc (NUMERIC(10,2)), devise (ENUM currency_enum), pdf_url (VARCHAR), emise_at (TIMESTAMP).

3.7. Notification Service

Responsabilité : Routage des alertes in-app et expédition de courriels transactionnels vers les clients (validation d'achat, facture émise, expédition) et commerçants (rupture de stock).

Notification Service – Spécification Technique**Points d'Entrée (APIs REST & Événements) :**

- GET /api/v1/notifications/my-notifications : Flux de notifications de l'utilisateur.
- PATCH /api/v1/notifications/{id}/read : Acquiescement d'une notification.
- **Événements consommés** : OrderStatusChangedEvent, StockAlertEvent, InvoiceGeneratedEvent.

Modèle de Données Associé :

- **Table notifications** : id (UUID, PK), user_id (UUID, FK → users), titre (VARCHAR), message (TEXT), type (ENUM : 'COMMANDE_STATUT', 'STOCK_BAS', 'PROMOTION', 'SYSTEME'), est_lu (BOOLEAN), created_at (TIMESTAMP).

3.8. Review Service

Responsabilité : Collecte des évaluations et commentaires clients après livraison du mobilier, modération éditoriale et publication des réponses du commerçant.

Review Service – Spécification Technique**Points d'Entrée (APIs REST) :**

- GET /api/v1/reviews/product/{productId} : Liste des avis validés d'un article.
- POST /api/v1/reviews : Dépôt d'un avis client (note sur 5, titre, commentaire).
- POST /api/v1/reviews/{id}/reply : Enregistrement d'une réponse officielle du commerçant.

Modèle de Données Associé :

- **Table reviews** : id (UUID, PK), user_id (UUID, FK → users), product_id (UUID, FK → products), titre (VARCHAR(150)), description (TEXT), note (INT, CHECK 1 à 5), reponse_commerçant (TEXT), repondu_at (TIMESTAMP), contrainte d'unicité (user_id, product_id).

3.9. Supplier Service

Responsabilité : Annuaire des fabricants partenaires, passation et suivi des commandes de réapprovisionnement (*Supply Orders*), et validation des réceptions de stocks.

Supplier Service – Spécification Technique

Points d'Entrée (APIs REST & Événements) :

- GET /api/v1/suppliers : Consultation du répertoire des fabricants.
- POST /api/v1/supply-orders : Création d'une commande d'achat fournisseur.
- PATCH /api/v1/supply-orders/{id}/receive : Réception de marchandises (émet l'événement SupplyReceivedEvent).
- **Événement consommé** : StockAlertEvent (suggestion automatique de commande d'achat).

Modèle de Données Associé :

- **Table fournisseurs** : id (UUID, PK), nom (VARCHAR), email (VARCHAR), telephone (VARCHAR), adresse (TEXT).
- **Table supply_orders** : id (UUID, PK), fournisseur_id (UUID, FK → fournisseurs), statut (ENUM : 'BROUILLON', 'EMISE', 'CONFIRMEE', 'EXPEDIEE', 'RECEPTIONNEE', 'ANNULEE'), montant_total (NUMERIC(10,2)), date_emission (TIMESTAMP), date_reception (TIMESTAMP).
- **Table supply_order_items** : id (UUID, PK), supply_order_id (UUID, FK → supply_orders), product_id (UUID, FK → products), quantite (INT), prix_unitaire (NUMERIC(10,2)).

3.10. Analytics & IA Batches (Spark)

Responsabilité : Calcul en différé des métriques business (chiffre d'affaires global et par article, produit star, taux de rupture) et calcul des recommandations par filtrage collaboratif.

Analytics & IA Batches – Spécification Technique**Exécution & Points d'Exposition :**

- Traitements batch Spark autonomes exécutés périodiquement (scheduler quotidien).
- Endpoints REST d'exposition vers le Backoffice (via API Gateway) :
 - GET /api/v1/kpi/global : Chiffre d'affaires consolidé, produit star, taux de rupture.
 - GET /api/v1/kpi/daily : Historique quotidien des ventes et du panier moyen.
 - GET /api/v1/recommendations/{productId} : Suggestions de produits similaires.

Modèle de Données Associé (Tables Pré-calculées) :

- **Table kpi_daily** : date (DATE, PK), ca_total (NUMERIC(12,2)), nb_commandes (INT), panier_moyen (NUMERIC(10,2)).
- **Table kpi_product** : product_id (UUID, PK), nom_produit (VARCHAR), quantite_vendue (INT), ca_produit (NUMERIC(12,2)), note_moyenne (NUMERIC(3,2)), stock_restant (INT), en_rupture (BOOLEAN).
- **Table kpi_global** : id (INT, PK), ca_global (NUMERIC(14,2)), nb_commandes_total (INT), produit_star_id (UUID, FK), taux_rupture (NUMERIC(5,2)).
- **Table product_recommendations** : product_id (UUID), recommended_product_id (UUID), score (NUMERIC(5,4)), PK composée (product_id, recommended_product_id).

4. Synthèse et Alignement avec le Projet

L'architecture microservices retenue pour Golden House garantit un alignement parfait entre les exigences pédagogiques et les meilleures pratiques de l'industrie :

1. **Alignement avec le cahier des charges et la base existante** : L'ensemble des modèles présentés dans ce rapport correspond rigoureusement au script SQL DDL opérationnel du projet (`backend/database/01-schema.sql`) et au jeu d'essai (`02-seed.sql`). Les types énumérés PostgreSQL, les contraintes d'intégrité et la gestion Stripe sont strictement respectés.
2. **Simplicité et robustesse du découpage** : En limitant les appels synchrones au strict chemin critique de l'achat (vérification du stock et initialisation bancaire), nous évitons les effets d'amplification de latence et les pannes en cascade. L'ensemble des opérations annexes (notifications, facturation, statistiques) est déporté en asynchrone.
3. **Intégration d'outils modernes (MCP)** : L'exposition d'une passerelle MCP dédiée permet aux agents conversationnels d'interroger directement les microservices pour guider l'acheteur sur mobile ou assister le gestionnaire sur le backoffice web sans complexifier les APIs transactionnelles.